FFT over Finite Fields?
Reed–Solomon error correction

Nasr-allah HITAR* Khalil BITAR

21 mars 2026

Résumé

Ce document présente une étude complète des codes de Reed-Solomon, une famille de codes correcteurs d'erreurs largement utilisée dans les systèmes de stockage et de transmission de données. Nous détaillons les fondements mathématiques basés sur l'arithmétique des corps finis $\mathrm{GF}(p^k)$, les méthodes d'encodage par évaluation polynomiale, et les algorithmes de décodage incluant le calcul des syndromes, l'algorithme de Berlekamp-Massey pour la résolution de l'équation clé, ainsi que les méthodes de localisation et d'évaluation des erreurs. Une implémentation pratique en C++ dans le cadre de la bibliothèque AFF3CT est également présentée, avec une analyse des performances et des considérations d'optimisation.

Mots-clés : Codes correcteurs, Reed-Solomon, corps finis, évaluation polynomiale, algorithme de Berlekamp-Massey, AFF3CT, codage canal.

*Encadré par Mr. Camille LEROUX

Table des matières

1 Introduction aux Corps de Galois et Extensions **4**
1.1 Corps de Galois $\mathbb{F}_2$ 4
1.2 Construction du corps $\mathbb{F}_{2^m}$ 5
1.2.1 Extensions de corps 5
1.2.2 Anneau de polynômes $\mathbb{F}_2[X]$ 5
1.2.3 Polynômes irréductibles 5
1.2.4 Construction de $\mathbb{F}_{2^m}$ comme corps quotient 5
1.2.5 Représentation par élément primitif 6
1.3 Polynômes Minimaux et Primitifs 7
1.3.1 Polynômes minimaux 7
1.3.2 Polynômes primitifs 7

2 Encodage Reed-Solomon **8**
2.1 Définition de la fonction d'encodage 8
2.1.1 Code en bloc linéaire 8
2.1.2 Définition formelle de l'encodage Reed-Solomon 8
2.1.3 Distance minimale et capacité de correction 9
2.2 Exemple introductif : RS(7, 4) 10
2.2.1 Paramètres du code 10
2.2.2 Table de représentation de $\mathbb{F}_{2^3}$ 10
2.2.3 Encodage d'un message 10

3 Décodage Reed-Solomon **12**
3.1 Polynôme d'erreur 12
3.2 Décodage par syndromes 12
3.3 Polynôme de localisation d'erreurs 13
3.3.1 Algorithme de Berlekamp-Massey 13
3.3.2 Recherche de Chien 14
3.3.3 Correction des erreurs et formule de Forney 15
3.4 Preuve de la Formule de Forney 15

4 FFT vs LFSR ? **18**
4.1 Encodeur LFSR classique 18
4.2 Pré-calcul des racines primitives 18
4.2.1 Encodeur FFT Reed-Solomon 18

5 Amélioration des performances par transformée de Fourier rapide (FFT) sur corps finis **19**
5.1 Principe de la transformée de Fourier sur un corps fini 19
5.2 Encodage systématique rapide par FFT 19
5.2.1 Interpolation par FFT 19
5.2.2 Division polynomiale rapide par FFT 20
5.3 Décodage systématique rapide par FFT 20
5.3.1 Calcul rapide des syndromes 20
5.3.2 Localisation rapide des erreurs 20
5.3.3 Calcul rapide des valeurs d'erreur 21
5.4 Implémentation pratique de la NTT 21

5.5 Analyse de complexité et gains 22

A Annexe : Tables de GF(8) **24**

1 Introduction aux Corps de Galois et Extensions

Les codes de Reed-Solomon reposent fondamentalement sur l'arithmétique des corps finis, également appelés corps de Galois. Cette section établit les bases théoriques nécessaires à la compréhension de ces structures algébriques.

1.1 Corps de Galois $\mathbb{F}_2$

Définition 1 (Corps)**.** *Un corps est un ensemble K muni de deux opérations internes, l'addition $+$ et la multiplication $\cdot$, tel que :*

(i) $(K, +)$ *est un groupe commutatif d'élément neutre noté* 0

(ii) $(K \setminus \{0\}, \cdot)$ *est un groupe commutatif d'élément neutre noté* 1

(iii) La multiplication est distributive sur l'addition : $\forall a, b, c \in K,\ a \cdot (b + c) = a \cdot b + a \cdot c$

Définition 2 (Corps de Galois $\mathbb{F}_2$)**.** *Le corps de Galois à deux éléments, noté $\mathbb{F}_2$, est l'ensemble $\{0, 1\}$ muni des opérations :*

— *Addition modulo 2 (XOR) :* $0 + 0 = 0,\ 0 + 1 = 1,\ 1 + 0 = 1,\ 1 + 1 = 0$

— *Multiplication modulo 2 (AND) :* $0 \cdot 0 = 0,\ 0 \cdot 1 = 0,\ 1 \cdot 0 = 0,\ 1 \cdot 1 = 1$

Proposition 1 (Propriétés fondamentales de $\mathbb{F}_2$)**.** *Le corps $\mathbb{F}_2$ possède les propriétés remarquables suivantes :*

1. ***Caractéristique 2*** *: Pour tout élément $a \in \mathbb{F}_2$, on a $a + a = 0$. En particulier, $1 + 1 = 0$, ce qui implique que $-1 = 1$.*
2. ***Auto-inverse additif*** *: Chaque élément est son propre opposé : $-a = a$ pour tout $a \in \mathbb{F}_2$.*
3. ***Idempotence*** *: L'addition et la soustraction sont identiques : $a - b = a + b$.*

Remarque 1. *La propriété $1+1 = 0$ dans $\mathbb{F}_2$ est fondamentale pour l'arithmétique binaire. Elle implique que l'addition binaire sans retenue correspond exactement à l'opération XOR au niveau des bits.*

Exemple 1. *Les tables de Cayley pour $\mathbb{F}_2$ sont :*

$+$	0	1
0	0	1
1	1	0

$\cdot$	0	1
0	0	0
1	0	1

1.2 Construction du corps $\mathbb{F}_{2^m}$

Le corps $\mathbb{F}_2$ ne contient que deux éléments, ce qui est insuffisant pour construire des codes de Reed-Solomon efficaces. Nous avons besoin d'extensions de corps pour obtenir suffisamment d'éléments distincts.

1.2.1 Extensions de corps

Remarque 2 (Nécessité des extensions)**.** *Pour un code de Reed-Solomon $RS(n, k)$ où n est la longueur du code et k la dimension de l'information, nous devons disposer d'au moins n éléments distincts dans notre corps. Puisque $\mathbb{F}_2$ ne contient que 2 éléments, nous devons construire des extensions pour obtenir 2^m éléments avec $m \geq 1$.*

1.2.2 Anneau de polynômes $\mathbb{F}_2[X]$

Définition 3 (Anneau de polynômes)**.** *L'anneau des polynômes à coefficients dans $\mathbb{F}_2$, noté $\mathbb{F}_2[X]$, est l'ensemble de tous les polynômes :*

$$P(X) = a_0 + a_1X + a_2X^2 + \cdots + a_nX^n, \quad a_i \in \mathbb{F}_2$$

muni de l'addition et de la multiplication usuelles des polynômes, avec coefficients réduits modulo 2.

Exemple 2. *Dans $\mathbb{F}_2[X]$, on a :*

$$(X^2 + X + 1) + (X^2 + 1) = (1 + 1)X^2 + X + (1 + 1) = X$$
$$(X + 1) \cdot (X + 1) = X^2 + X + X + 1 = X^2 + 1$$

1.2.3 Polynômes irréductibles

Définition 4 (Polynôme irréductible)**.** *Un polynôme $P(X) \in \mathbb{F}_2[X]$ de degré ≥ 1 est dit* ***irréductible*** *sur $\mathbb{F}_2$ s'il ne peut pas être factorisé en un produit de deux polynômes de degré strictement inférieur, c'est-à-dire :*

$$P(X) = A(X) \cdot B(X) \implies \deg A = 0 \text{ ou } \deg B = 0$$

Proposition 2. *Un polynôme irréductible sur $\mathbb{F}_2$ joue le même rôle qu'un nombre premier dans $\mathbb{Z}$: il permet de construire un corps quotient.*

Exemple 3 (Polynômes irréductibles de petits degrés)**.**

- *Degré 1 : tout polynôme.*
- *Degré 2 : $X^2 + X + 1$ (le seul)*
- *Degré 3 : $X^3 + X + 1$, $X^3 + X^2 + 1$*
- *Degré 4 : $X^4 + X + 1$, $X^4 + X^3 + 1$, $X^4 + X^3 + X^2 + X + 1$*

1.2.4 Construction de $\mathbb{F}_{2^m}$ comme corps quotient

Théorème 1 (Construction de $\mathbb{F}_{2^m}$)**.** *Soit $P(X)$ un polynôme irréductible de degré m sur $\mathbb{F}_2$. Alors le quotient*

$$\mathbb{F}_{2^m} = \mathbb{F}_2[X]/(P(X))$$

est un corps contenant exactement 2^m éléments.

Esquisse de démonstration.

1. Chaque élément de $\mathbb{F}_{2^m}$ peut être représenté de manière unique par un polynôme de degré $< m$ à coefficients dans $\mathbb{F}_2$.
2. Il y a donc 2^m tels polynômes (2 choix pour chacun des m coefficients).
3. L'irréductibilité de $P(X)$ garantit que tout élément non nul possède un inverse, faisant de $\mathbb{F}_{2^m}$ un corps.

□

Exemple 4 (Construction de $\mathbb{F}_{2^3}$ avec $P(X) = X^3+X+1$)**.** *Le corps* $\mathbb{F}_{2^3} = \mathbb{F}_2[X]/(X^3+X+1)$ *contient* $2^3 = 8$ *éléments, représentés par :*

$$\{0, 1, X, X+1, X^2, X^2+1, X^2+X, X^2+X+1\}$$

L'addition se fait coefficient par coefficient modulo 2, et la multiplication se fait modulo $P(X) = X^3 + X + 1$.

Par exemple, calculons $(X^2+1) \cdot X$ *:*

$$\begin{aligned}(X^2+1) \cdot X &= X^3 + X \\ &\equiv X + 1 + X \pmod{X^3+X+1} \quad \textit{(car } X^3 \equiv X+1\textit{)} \\ &= 1\end{aligned}$$

1.2.5 Représentation par élément primitif

Définition 5 (Élément primitif)**.** *Un élément* $\alpha \in \mathbb{F}_{2^m}$ *est dit* ***primitif*** *si son ordre multiplicatif est* $2^m - 1$*, c'est-à-dire si :*

$$\{\alpha^0, \alpha^1, \alpha^2, \ldots, \alpha^{2^m-2}\} = \mathbb{F}_{2^m} \setminus \{0\}$$

Théorème 2. *Pour tout corps fini* $\mathbb{F}_{2^m}$*, il existe au moins un élément primitif. De plus, si* $P(X)$ *est un polynôme primitif de degré* m *et* α *une racine de* $P(X)$ *dans* $\mathbb{F}_{2^m}$*, alors* α *est un élément primitif.*

Exemple 5 (Représentation de $\mathbb{F}_{2^3}$ par puissances de α)**.** *Soit* α *une racine de* $P(X) = X^3 + X + 1$*. Alors* $\alpha^3 = \alpha + 1$*, et on peut calculer :*

$$\begin{aligned}\alpha^0 &= 1 \\ \alpha^1 &= \alpha \\ \alpha^2 &= \alpha^2 \\ \alpha^3 &= \alpha + 1 \\ \alpha^4 &= \alpha^2 + \alpha \\ \alpha^5 &= \alpha^2 + \alpha + 1 \\ \alpha^6 &= \alpha^2 + 1 \\ \alpha^7 &= 1\end{aligned}$$

On constate que $\alpha^7 = 1$*, confirmant que* α *est primitif d'ordre* $2^3 - 1 = 7$.

1.3 Polynômes Minimaux et Primitifs

1.3.1 Polynômes minimaux

Définition 6 (Polynôme minimal). *Soit $\beta \in \mathbb{F}_{2^m}$. Le **polynôme minimal** de β sur $\mathbb{F}_2$, noté $m_\beta(X)$, est le polynôme unitaire de plus petit degré dans $\mathbb{F}_2[X]$ tel que $m_\beta(\beta) = 0$.*

Proposition 3 (Propriétés des polynômes minimaux).

1. *Le polynôme minimal $m_\beta(X)$ est irréductible sur $\mathbb{F}_2$.*
2. *Le degré de $m_\beta(X)$ divise m (où $\beta \in \mathbb{F}_{2^m}$).*
3. *$m_\beta(X)$ divise tout polynôme $Q(X) \in \mathbb{F}_2[X]$ tel que $Q(\beta) = 0$.*
4. *Si $\beta = \alpha^i$ où α est primitif, alors $m_{\alpha^i}(X) = m_{\alpha^j}(X)$ si et seulement si i et j sont dans la même classe cyclotomique modulo $2^m - 1$.*

Définition 7 (Classe cyclotomique). *La classe cyclotomique de i modulo $n = 2^m - 1$ est l'ensemble :*

$$C_i = \{i \cdot 2^j \bmod n : j \geq 0\}$$

Exemple 6 (Classes cyclotomiques pour $m = 3$). *Pour $m = 3$, on a $n = 2^3 - 1 = 7$. Les classes cyclotomiques sont :*

$$\begin{aligned}
C_0 &= \{0\} \\
C_1 &= \{1, 2, 4\} \\
C_3 &= \{3, 6, 5\} \\
C_7 &= \{0\} \quad (\textit{car } 7 \equiv 0 \pmod 7)
\end{aligned}$$

Théorème 3 (Construction du polynôme minimal). *Le polynôme minimal de α^i où α est primitif dans $\mathbb{F}_{2^m}$ est donné par :*

$$m_{\alpha^i}(X) = \prod_{j \in C_i} (X - \alpha^j)$$

où le produit est pris sur tous les éléments de la classe cyclotomique C_i.

Exemple 7 (Polynômes minimaux dans $\mathbb{F}_{2^3}$). *Pour α primitif dans $\mathbb{F}_{2^3}$ (racine de $X^3 + X + 1$) :*

$$\begin{aligned}
m_{\alpha^0}(X) = m_1(X) &= X + 1 \\
m_{\alpha^1}(X) = m_{\alpha^2}(X) = m_{\alpha^4}(X) &= (X - \alpha)(X - \alpha^2)(X - \alpha^4) = X^3 + X + 1 \\
m_{\alpha^3}(X) = m_{\alpha^5}(X) = m_{\alpha^6}(X) &= (X - \alpha^3)(X - \alpha^5)(X - \alpha^6) = X^3 + X^2 + 1
\end{aligned}$$

1.3.2 Polynômes primitifs

Définition 8 (Polynôme primitif). *Un polynôme irréductible $P(X)$ de degré m sur $\mathbb{F}_2$ est dit **primitif** si l'ordre de toute racine α de $P(X)$ dans $\mathbb{F}_{2^m}$ est exactement $2^m - 1$.*

Proposition 4 (Caractérisation des polynômes primitifs). *Un polynôme irréductible $P(X)$ de degré m est primitif si et seulement si $P(X)$ divise $X^{2^m-1} + 1$ mais ne divise aucun $X^k + 1$ pour $1 \leq k < 2^m - 1$.*

Remarque 3 (Importance pour Reed-Solomon)**.** *Les polynômes primitifs sont essentiels pour la construction des codes de Reed-Solomon car ils garantissent l'existence d'un élément générateur α tel que toutes les puissances $\alpha^0, \alpha^1, \ldots, \alpha^{n-1}$ sont distinctes, permettant ainsi de définir les positions d'évaluation du code.*

Exemple 8 (Test de primitivité)**.** *Pour $m = 3$, testons si $P(X) = X^3 + X + 1$ est primitif :*

- $2^m - 1 = 7$.
- *Il faut vérifier que $P(X)$ divise $X^7 + 1$ mais pas $X^k + 1$ pour $k \in \{1, 2, 3, 4, 5, 6\}$.*
- *On a déjà vu que si α est racine de $P(X)$, alors $\alpha^7 = 1$, donc $P(X)$ divise $X^7 + 1$.*
- *Par calcul direct, $P(X)$ ne divise pas $X^k + 1$ pour $k < 7$.*
- *Donc $P(X) = X^3 + X + 1$ est primitif.* □

2 Encodage Reed-Solomon

2.1 Définition de la fonction d'encodage

2.1.1 Code en bloc linéaire

Définition 9 (Code en bloc)**.** *Un code en bloc (n, k) sur un alphabet $\mathcal{A}$ est un sous-ensemble $\mathcal{C} \subseteq \mathcal{A}^n$ de dimension k, où :*

- *n est la longueur du code (nombre de symboles en sortie)*
- *k est la dimension de l'information (nombre de symboles d'information)*
- *$\mathcal{A}$ est l'alphabet (pour Reed-Solomon, $\mathcal{A} = \mathbb{F}_{2^m}$)*

Définition 10 (Code linéaire)**.** *Un code $\mathcal{C}$ est dit **linéaire** s'il forme un sous-espace vectoriel de $\mathcal{A}^n$, c'est-à-dire :*

$$\forall \mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C},\ \forall \lambda \in \mathcal{A}, \quad \mathbf{c}_1 + \mathbf{c}_2 \in \mathcal{C} \text{ et } \lambda \mathbf{c}_1 \in \mathcal{C}$$

2.1.2 Définition formelle de l'encodage Reed-Solomon

Définition 11 (Polynôme générateur)**.** *Le **polynôme générateur** d'un code $RS(n, k)$ avec $2t = n - k$ est le polynôme de degré $2t$ dont les racines sont $2t$ puissances consécutives de l'élément primitif α :*

$$g(X) = \prod_{i=b}^{b+2t-1} (X - \alpha^i)$$

Tout mot de code valide est un multiple de $g(X)$ dans $\mathbb{F}_{2^m}[X]$. (On prend normalement $b = 1$ pour des codes Reed-Solomon.)

Définition 12 (Encodage systématique $\mathrm{RS}(n, k)$)**.** *Soit $\mathbf{d} = (d_0, d_1, \ldots, d_{k-1}) \in \mathbb{F}_{2^m}^k$ le message d'information et*

$$D(X) = d_0 + d_1 X + \cdots + d_{k-1} X^{k-1}$$

son polynôme associé. L'encodage systématique se déroule en trois étapes :

Étape 1 — Décalage. *On multiplie $D(X)$ par X^{2t} afin de réserver les $2t$ positions de plus bas degré aux symboles de parité :*

$$\tilde{D}(X) = D(X) \cdot X^{2t}$$

Étape 2 — Division euclidienne. *On divise $\tilde{D}(X)$ par $g(X)$:*

$$\tilde{D}(X) = Q(X) \cdot g(X) + R(X), \quad \deg R < 2t$$

Le reste $R(X) = r_0 + r_1 X + \cdots + r_{2t-1} X^{2t-1}$ constitue les ***symboles de parité****.*

Étape 3 — Construction du mot de code. *On soustrait (i.e. ajoute en caractéristique 2) le reste au polynôme décalé :*

$$C(X) = \tilde{D}(X) - R(X) = D(X) \cdot X^{2t} + R(X)$$

Le vecteur de code est alors :

$$\mathbf{c} = \big(r_0,\, r_1,\, \ldots,\, r_{2t-1},\, d_0,\, d_1,\, \ldots,\, d_{k-1}\big)$$

Proposition 5. *Le polynôme $C(X)$ ainsi construit est bien un multiple de $g(X)$, et les k symboles de plus haut degré du mot de code coïncident exactement avec les symboles d'information.*

Démonstration. Par construction, $C(X) = \tilde{D}(X) - R(X) = Q(X) \cdot g(X)$, donc $g(X) \mid C(X)$. De plus, les coefficients de $X^{2t}, \ldots, X^{n-1}$ dans $C(X)$ sont exactement ceux de $D(X) \cdot X^{2t}$, soit $d_0, \ldots, d_{k-1}$: le code est bien systématique. □

Remarque 4.

- ***Systématique*** *: les k derniers symboles du mot de code sont le message original, ce qui facilite le décodage et la lecture directe de l'information.*
- ***Redondance*** *: les $2t$ premiers symboles sont les symboles de parité calculés par division polynomiale ; ils permettent de corriger jusqu'à $t = \lfloor (n-k)/2 \rfloor$ erreurs.*

Théorème 4 (Linéarité du code RS)**.** *Le code de Reed-Solomon RS(n, k) est un code linéaire de dimension k sur $\mathbb{F}_{2^m}$.*

Démonstration. Pour montrer que $\mathrm{RS}(n, k)$ est linéaire, il suffit de vérifier que l'application Φ est linéaire :

$$\begin{aligned}
\Phi(\mathbf{d}_1 + \mathbf{d}_2) &= (D_1(\alpha^i) + D_2(\alpha^i))_{i=0}^{n-1} \\
&= (D_1(\alpha^i))_{i=0}^{n-1} + (D_2(\alpha^i))_{i=0}^{n-1} \\
&= \Phi(\mathbf{d}_1) + \Phi(\mathbf{d}_2)
\end{aligned}$$

De même pour la multiplication scalaire. Donc $\mathrm{RS}(n, k)$ est un sous-espace vectoriel de $\mathbb{F}_{2^m}^n$. □

2.1.3 Distance minimale et capacité de correction

Définition 13 (Distance de Hamming)**.** *La distance de Hamming entre deux mots $\mathbf{c}_1, \mathbf{c}_2 \in \mathbb{F}_{2^m}^n$ est :*

$$d_H(\mathbf{c}_1, \mathbf{c}_2) = |\{i : c_{1,i} \neq c_{2,i}\}|$$

Définition 14 (Distance minimale)**.** *La distance minimale d'un code $\mathcal{C}$ est :*

$$d_{\min} = \min_{\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C},\, \mathbf{c}_1 \neq \mathbf{c}_2} d_H(\mathbf{c}_1, \mathbf{c}_2)$$

Théorème 5 (Distance minimale du code RS)**.** *Pour un code de Reed-Solomon RS(n, k), la distance minimale est :*

$$d_{\min} = n - k + 1$$

Esquisse. Puisque les mots de code correspondent à des polynômes de degré $< k$ évalués en n points, deux polynômes distincts peuvent coïncider en au plus $k - 1$ points. Donc ils diffèrent en au moins $n - (k - 1) = n - k + 1$ points. □

Corollaire 1 (Capacité de correction)**.** *Un code RS(n, k) peut corriger jusqu'à :*

$$t = \left\lfloor \frac{n-k}{2} \right\rfloor$$

erreurs de symboles.

2.2 Exemple introductif : RS(7, 4)

Nous allons construire en détail le code Reed-Solomon RS$(7, 4)$ sur $\mathbb{F}_{2^3}$.

2.2.1 Paramètres du code

- **Longueur du code** : $n = 7$
- **Dimension de l'information** : $k = 4$
- **Redondance** : $n - k = 3$
- **Corps de base** : $\mathbb{F}_{2^3} = \mathbb{F}_2[X]/(X^3 + X + 1)$ avec $|\mathbb{F}_{2^3}| = 2^3 = 8$ éléments
- **Polynôme irréductible** : $P(X) = X^3 + X + 1$ (primitif)
- **Élément primitif** : α racine de $P(X)$, avec $\alpha^7 = 1$
- **Capacité de correction** : $t = \lfloor 3/2 \rfloor = 1$ erreur

2.2.2 Table de représentation de $\mathbb{F}_{2^3}$

Voir Annexe A : Tables de GF(8).

2.2.3 Encodage d'un message

Considérons le message d'information $\mathbf{d} = (d_0, d_1, d_2, d_3) = (1, \alpha, \alpha^2, \alpha^3)$.
Étape 1 : Construction du polynôme d'information

$$D(X) = d_0 + d_1 X + d_2 X^2 + d_3 X^3 = 1 + \alpha X + \alpha^2 X^2 + \alpha^3 X^3$$

Étape 2 : Évaluation aux points α^i, $i = 0, 1, \ldots, 6$

$$\begin{aligned} c_0 = D(\alpha^0) = D(1) &= 1 + \alpha + \alpha^2 + \alpha^3 \\ &= 1 + \alpha + \alpha^2 + (\alpha + 1) \\ &= \alpha^2 \quad \text{(en utilisant } 1 + 1 = 0 \text{ dans } \mathbb{F}_2) \end{aligned}$$

$$
\begin{aligned}
c_1 = D(\alpha^1) = D(\alpha) &= 1 + \alpha \cdot \alpha + \alpha^2 \cdot \alpha^2 + \alpha^3 \cdot \alpha^3 \\
&= 1 + \alpha^2 + \alpha^4 + \alpha^6 \\
&= 1 + \alpha^2 + (\alpha^2 + \alpha) + (\alpha^2 + 1) \\
&= \alpha + \alpha^2 = \alpha^4
\end{aligned}
$$

$$
\begin{aligned}
c_2 = D(\alpha^2) &= 1 + \alpha \cdot \alpha^2 + \alpha^2 \cdot \alpha^4 + \alpha^3 \cdot \alpha^6 \\
&= 1 + \alpha^3 + \alpha^6 + \alpha^9 \\
&= 1 + \alpha^3 + \alpha^6 + \alpha^2 \quad (\text{car } \alpha^9 = \alpha^{9 \bmod 7} = \alpha^2)
\end{aligned}
$$

De même pour c_3, c_4, c_5, c_6 (calculs omis pour la concision).
Le mot de code complet est :

$$
\mathbf{c} = (c_0, c_1, c_2, c_3, c_4, c_5, c_6)
$$

Remarque 5 (Complexité de calcul)**.** *L'évaluation directe nécessite $O(nk)$ opérations dans $\mathbb{F}_{2^m}$. Des méthodes plus efficaces existent, notamment l'utilisation de la transformée de Fourier rapide (FFT) sur les corps finis.*

3 Décodage Reed-Solomon

Le décodage d'un code Reed-Solomon consiste à retrouver le mot de code original à partir d'un mot reçu potentiellement corrompu. Dans la vue BCH, un mot de code $\mathbf{c}$ de longueur $n = 2^m - 1$ sur $\mathbb{F}_{2^m}$ est défini par ses valeurs en des points $\alpha^0, \alpha^1, \ldots, \alpha^{n-1}$, où α est un élément primitif. Le décodeur exploite les contraintes algébriques imposées par les racines du polynôme générateur pour localiser et corriger les erreurs.

On suppose qu'au plus t erreurs se sont produites lors de la transmission, où $t = \lfloor (d_{\min} - 1)/2 \rfloor$ et $d_{\min} = n - k + 1$ est la distance minimale du code. Le mot reçu s'écrit :

$$\mathbf{r} = \mathbf{c} + \mathbf{e}$$

où $\mathbf{e} = (e_0, e_1, \ldots, e_{n-1})$ est le vecteur d'erreur, dont au plus t composantes sont non nulles.

Le processus de décodage BCH se décompose en trois étapes successives :

1. Calcul du *syndrome* à partir du mot reçu.
2. Détermination du *polynôme de localisation d'erreurs* à partir des syndromes.
3. Calcul des *valeurs d'erreurs* aux positions identifiées, puis correction du mot reçu.

3.1 Polynôme d'erreur

On modélise le vecteur d'erreur $\mathbf{e}$ par un polynôme :

$$E(X) = \sum_{i=0}^{n-1} e_i X^i$$

Soit $\nu \leq t$ le nombre d'erreurs effectivement survenues. On note $i_1, i_2, \ldots, i_\nu$ les *positions* des erreurs (les indices i tels que $e_i \neq 0$) et $Y_j = e_{i_j}$ les *valeurs* des erreurs en ces positions. En posant $X_j = \alpha^{i_j}$ le *localisateur* de la j-ième erreur, on obtient :

$$E(\alpha^l) = \sum_{j=1}^{\nu} Y_j X_j^l, \quad l = b, \ldots, b + 2t - 1$$

Le mot reçu s'exprime comme $R(X) = C(X) + E(X)$. Puisque $C(\alpha^l) = 0$ pour $l = b, b+1, \ldots, b+2t-1$ (racines du polynôme générateur), on a :

$$R(\alpha^l) = E(\alpha^l) = \sum_{j=1}^{\nu} Y_j X_j^l$$

Ces quantités sont précisément les syndromes du mot reçu.

3.2 Décodage par syndromes

Définition 15 (Syndrome)**.** *Soit* $\mathbf{r}$ *un mot reçu de polynôme associé* $R(X)$*. Pour* $l = b, b+1, \ldots, b+2t-1$*, le* ***syndrome*** S_l *est défini par :*

$$S_l = R(\alpha^l) = \sum_{i=0}^{n-1} r_i \, \alpha^{li}$$

Si $\mathbf{r}$ *est un mot de code valide, alors* $S_l = 0$ *pour tout* l*. En présence d'erreurs, on obtient :*

$$S_l = E(\alpha^l) = \sum_{j=1}^{\nu} Y_j X_j^l$$

Les $2t$ *syndromes* $S_b, S_{b+1}, \ldots, S_{b+2t-1}$ *constituent le* ***vecteur de syndrome****, noté* $\mathbf{S}$*.*

Le calcul des syndromes est la première étape concrète du décodage : il suffit d'évaluer $R(X)$ en chacune des $2t$ racines du polynôme générateur.

Algorithm 1 Calcul du vecteur de syndrome

```
for l = b to b + 2t − 1 do
    S_l ← 0
    for i = 0 to n − 1 do
        S_l ← S_l + r_i · α^{li}
    end for
end for
```

3.3 Polynôme de localisation d'erreurs

Définition 16 (Polynôme de localisation d'erreurs)**.** *Le* ***polynôme de localisation d'erreurs*** $\Lambda(X)$ *est le polynôme dont les racines sont les inverses des localisateurs* $X_j = \alpha^{i_j}$ *:*

$$\Lambda(X) = \prod_{j=1}^{\nu}(1 - X_j X) = 1 + \Lambda_1 X + \Lambda_2 X^2 + \cdots + \Lambda_\nu X^\nu$$

Ainsi $\Lambda(X_j^{-1}) = 0$ *pour chaque position d'erreur* i_j*. Connaître* $\Lambda(X)$ *permet de retrouver toutes les positions d'erreurs.*

Les coefficients $\Lambda_1, \ldots, \Lambda_\nu$ vérifient les *équations clés de Newton*, qui relient syndromes et localisateurs :

$$S_{l+\nu} + \Lambda_1 S_{l+\nu-1} + \cdots + \Lambda_\nu S_l = 0, \quad l = b, \ldots, b + t - 1$$

Cette relation peut s'écrire sous forme matricielle :

$$\begin{pmatrix} S_b & S_{b+1} & \cdots & S_{b+\nu-1} \\ S_{b+1} & S_{b+2} & \cdots & S_{b+\nu} \\ \vdots & & \ddots & \vdots \\ S_{b+\nu-1} & S_{b+\nu} & \cdots & S_{b+2\nu-2} \end{pmatrix} \begin{pmatrix} \Lambda_\nu \\ \Lambda_{\nu-1} \\ \vdots \\ \Lambda_1 \end{pmatrix} = - \begin{pmatrix} S_{b+\nu} \\ S_{b+\nu+1} \\ \vdots \\ S_{b+2\nu-1} \end{pmatrix}$$

3.3.1 Algorithme de Berlekamp-Massey

L'algorithme de Berlekamp-Massey calcule $\Lambda(X)$ de façon itérative à partir des syndromes, en cherchant le registre à décalage linéaire de longueur minimale qui les génère.

Algorithm 2 Berlekamp-Massey

$\Lambda(X) \leftarrow 1, \quad B(X) \leftarrow 1, \quad L \leftarrow 0, \quad m \leftarrow 1$
for $r = 0$ **to** $2t - 1$ **do**
  $\delta \leftarrow S_{b+r} + \sum_{i=1}^{L} \Lambda_i \cdot S_{b+r-i}$
  if $\delta = 0$ **then**
    $m \leftarrow m + 1$
  else if $2L \leq r$ **then**
    $T(X) \leftarrow \Lambda(X)$
    $\Lambda(X) \leftarrow \Lambda(X) - \delta \cdot X^m \cdot B(X)$
    $L \leftarrow r + 1 - L, \quad B(X) \leftarrow T(X), \quad m \leftarrow 1$
  else
    $\Lambda(X) \leftarrow \Lambda(X) - \delta \cdot X^m \cdot B(X)$
    $m \leftarrow m + 1$
  end if
end for

3.3.2 Recherche de Chien

Une fois $\Lambda(X)$ obtenu, on identifie les positions d'erreurs par la **recherche de Chien** : on évalue $\Lambda(X)$ en chaque élément α^{-i} pour $i = 0, \ldots, n-1$. Si $\Lambda(\alpha^{-i}) = 0$, alors la position i est une position d'erreur.

$\mathcal{E}$ est l'ensemble des positions d'erreurs. Dans cet algorithme, on initialise cet ensemble à l'ensemble vide et on ajoute une position d'erreur chaque fois qu'on trouve $\Lambda(\alpha^{-i}) = 0$

Algorithm 3 Recherche de Chien

$\mathcal{E} \leftarrow \emptyset$
for $i = 0$ **to** $n - 1$ **do**
  if $\Lambda(\alpha^{-i}) = 0$ **then**
    $\mathcal{E} \leftarrow \mathcal{E} \cup \{i\}$
  end if
end for

Algorithm 4 Recherche de Chien sans Horner

Soit $\Lambda(x) = \sum_{j=0}^{L} \Lambda_j x^j$ le polynôme localisateur d'erreurs de degré L.
Précalculer les constantes $\beta_j = \alpha^{-j}$ pour $j = 0, 1, \ldots, L$ (avec $\beta_0 = 1$).
Initialiser un tableau $R[0..L]$ avec $R[j] = \Lambda_j$ pour $j = 0, \ldots, L$.
$\mathcal{E} \leftarrow \emptyset$
for $i = 0$ **to** $n - 1$ **do**
    $S \leftarrow \bigoplus_{j=0}^{L} R[j]$ ▷ Addition dans $\mathbb{F}_{2^m}$ (XOR)
    if $S = 0$ **then**
        $\mathcal{E} \leftarrow \mathcal{E} \cup i$
    end if
    for $j = 1$ **to** L **do**
        $R[j] \leftarrow R[j] \cdot \beta_j$ ▷ Mise à jour pour $i + 1$
    end for
end for
return $\mathcal{E}$

3.3.3 Correction des erreurs et formule de Forney

Dans le cas binaire ($\mathbb{F}_2$), toute valeur d'erreur non nulle vaut nécessairement 1. La correction est alors immédiate : pour chaque position $i \in \mathcal{E}$, on pose $\hat{r}_i = r_i + 1$.

Remarque 6 (Géneralisation)**.** *Dans le cas général sur $\mathbb{F}_q$, les valeurs d'erreur Y_j sont calculées par la* ***formule de Forney*** *:*

$$Y_j = -\frac{X_j^{1-b}\,\Omega(X_j^{-1})}{\Lambda'(X_j^{-1})}$$

où $\Omega(X) = S(X) \cdot \Lambda(X) \bmod X^{2t}$ est le ***polynôme évaluateur d'erreurs****, avec $S(X) = \sum_{l=b}^{b+2t-1} S_l X^{l-b}$, et Λ' désigne la dérivée formelle de Λ.*

3.4 Preuve de la Formule de Forney

1. Définitions Rappelons les notations utilisées par la suite :

— Le polynôme localisateur d'erreurs : $\Lambda(X) = \prod_{j=1}^{\nu}(1 - X_j X)$.
— Le syndrome : $S_l = \sum_{j=1}^{\nu} Y_j X_j^l$ pour $l = b, \ldots, b + 2t - 1$.
— Le polynôme des syndromes : $S(X) = \sum_{l=b}^{b+2t-1} S_l X^{l-b}$.
— Le polynôme évaluateur d'erreurs : $\Omega(X) \equiv S(X) \cdot \Lambda(X) \pmod{X^{2t}}$.

2. Développement du polynôme des syndromes En substituant l'expression de S_l dans $S(X)$, on obtient :

$$S(X) = \sum_{l=b}^{b+2t-1} \left(\sum_{j=1}^{\nu} Y_j X_j^l \right) X^{l-b}$$

En réorganisant simplement les sommes :

$$S(X) = \sum_{j=1}^{\nu} Y_j X_j^b \sum_{l=0}^{2t-1} (X_j X)^l$$

En utilisant le développement en série entière $(1-z)^{-1} = \sum z^k$, on obtient la congruence modulo X^{2t} :

$$S(X) \equiv \sum_{j=1}^{\nu} \frac{Y_j X_j^b}{1 - X_j X} \pmod{X^{2t}}$$

3. Construction du polynôme évaluateur $\Omega(X)$ On multiplie $S(X)$ par $\Lambda(X)$:

$$\Omega(X) \equiv \left(\sum_{j=1}^{\nu} \frac{Y_j X_j^b}{1 - X_j X} \right) \prod_{i=1}^{\nu} (1 - X_i X) \pmod{X^{2t}}$$

Le terme $(1-X_jX)$ au dénominateur s'annule avec le terme correspondant dans le produit $\Lambda(X)$. Puisque le degré résultant est strictement inférieur à $2t$, la congruence devient une égalité stricte :

$$\Omega(X) = \sum_{j=1}^{\nu} Y_j X_j^b \prod_{i \neq j} (1 - X_i X)$$

4. Évaluation à l'inverse d'une racine ($X = X_k^{-1}$) Pour isoler l'erreur Y_k à la position X_k, on évalue $\Omega(X)$ en X_k^{-1}. Pour tout $j \neq k$, le produit contiendra le facteur $(1-X_kX_k^{-1}) = 1-1 = 0$. Toute la somme s'annule donc, à l'exception du terme où $j = k$:

$$\Omega(X_k^{-1}) = Y_k X_k^b \prod_{i \neq k} (1 - X_i X_k^{-1})$$

5. Utilisation de la dérivée formelle Calculons la dérivée formelle de $\Lambda(X)$ par rapport à X en utilisant la règle du produit :

$$\Lambda'(X) = \sum_{j=1}^{\nu} (-X_j) \prod_{i \neq j} (1 - X_i X)$$

En évaluant cette dérivée en X_k^{-1}, tous les termes de la somme s'annulent (pour la même raison qu'à l'étape 4) sauf pour $j = k$:

$$\Lambda'(X_k^{-1}) = -X_k \prod_{i \neq k} (1 - X_i X_k^{-1})$$

On peut alors isoler le produit résiduel :

$$\prod_{i \neq k} (1 - X_i X_k^{-1}) = -\frac{\Lambda'(X_k^{-1})}{X_k}$$

6. Résolution finale pour Y_k On substitue ce résultat dans l'équation de l'étape 4 :

$$\Omega(X_k^{-1}) = Y_k X_k^b \left(-\frac{\Lambda'(X_k^{-1})}{X_k} \right)$$
$$\Omega(X_k^{-1}) = -Y_k X_k^{b-1} \Lambda'(X_k^{-1})$$

Il ne reste plus qu'à isoler l'amplitude de l'erreur Y_k pour obtenir la formule finale de Forney :

$$Y_k = -\frac{X_k^{1-b}\Omega(X_k^{-1})}{\Lambda'(X_k^{-1})}$$

Remarque : Dans un corps de caractéristique 2 comme $\mathbb{F}_{2^m}$, l'addition et la soustraction sont équivalentes, ce qui permet d'ignorer le signe négatif lors de l'implémentation.

4 FFT vs LFSR ?

4.1 Encodeur LFSR classique

L'encodeur classique utilise un *registre à décalage linéaire* (LFSR) :

$$\text{feedback} = m_i + p_{r-1}$$

$$p_j = p_{j-1} + \text{feedback} \cdot g_j$$

avec $r = n - k$ et g_j les coefficients du polynôme générateur.

$$\text{Complexité} = O(k \cdot r)$$

Cette méthode est simple, mais la dépendance sur le registre rend difficile la vectorisation et le parallélisme.

—

4.2 Pré-calcul des racines primitives

Pour des champs $GF(2^m)$ avec $m \leq 20$, on peut pré-calculer les racines primitives α^i pour accélérer la FFT et d'autres algorithmes sur le champ :

$$\text{primitive_roots}[m][i] = \alpha^i$$

Cela permet d'accélérer :

- La FFT sur GF
- Le calcul des polynômes générateurs
- La multiplication de polynômes
-

4.2.1 Encodeur FFT Reed-Solomon

Un code RS peut aussi être vu comme l'évaluation d'un polynôme en n points du champ :

$$C_i = M(\alpha^i), \quad i = 0, \ldots, n-1$$

La FFT sur corps finis permet de calculer rapidement ces évaluations avec une complexité :

$$O(n \log n)$$

5 Amélioration des performances par transformée de Fourier rapide (FFT) sur corps finis

Les méthodes classiques d'encodage et de décodage des codes de Reed-Solomon ont une complexité quadratique en la longueur du code, ce qui peut devenir un goulot d'étranglement pour les applications à haut débit. Une alternative élégante et efficace consiste à utiliser la transformée de Fourier discrète sur les corps finis, également appelée transformée nombre-théorique (NTT), qui permet d'atteindre une complexité quasi-linéaire $O(n \log n)$ pour l'encodage et le décodage. Nous présentons ici comment adapter ces techniques pour obtenir un encodage et un décodage systématiques.

5.1 Principe de la transformée de Fourier sur un corps fini

Soit $\mathbb{F}_{2^q}$ un corps fini de caractéristique p (avec $q = p^m$). On suppose qu'il existe un élément primitif α d'ordre n, c'est-à-dire $\alpha^n = 1$ et les puissances $\alpha^0, \alpha^1, \ldots, \alpha^{n-1}$ sont toutes distinctes. Pour un vecteur $\mathbf{a} = (a_0, a_1, \ldots, a_{n-1}) \in \mathbb{F}_{2^q}^n$, on définit sa transformée de Fourier (ou NTT) $\mathbf{A} = (A_0, A_1, \ldots, A_{n-1})$ par :

$$A_k = \sum_{i=0}^{n-1} a_i \, \alpha^{ik}, \quad k = 0, 1, \ldots, n-1.$$

La transformée inverse est donnée par :

$$a_i = \frac{1}{n} \sum_{k=0}^{n-1} A_k \, \alpha^{-ik},$$

où $1/n$ désigne l'inverse multiplicatif de n dans $\mathbb{F}_{2^q}$. Cette transformée possède les propriétés classiques de la DFT : linéarité, convolution, et elle peut être calculée rapidement par un algorithme de type Cooley-Tukey lorsque n est une puissance de deux.

5.2 Encodage systématique rapide par FFT

L'encodage systématique d'un code $\mathrm{RS}(n, k)$ consiste, à partir d'un message $\mathbf{d} = (d_0, \ldots, d_{k-1})$, à construire un mot de code $\mathbf{c} = (p_0, \ldots, p_{r-1}, d_0, \ldots, d_{k-1})$ avec $r = n - k$ symboles de parité. Dans la représentation polynomiale, cela revient à trouver le polynôme $C(X)$ de degré $< n$ tel que $C(X) \equiv D(X)X^r \pmod{g(X)}$, où $g(X)$ est le polynôme générateur de degré r. Une méthode utilisant la FFT repose sur l'interpolation et l'évaluation rapides.

5.2.1 Interpolation par FFT

On souhaite construire un polynôme $P(X)$ de degré $< n$ qui vérifie $P(\alpha^i) = d_i$ pour $i = 0, \ldots, k-1$ (positions d'information) et $P(\alpha^j) = 0$ pour $j = k, \ldots, n-1$ (positions de parité). Alors le mot de code systématique est simplement l'évaluation de P en tous les points α^i. L'interpolation peut être réalisée efficacement en $O(n \log^2 n)$ en utilisant la FFT pour les divisions polynomiales (algorithme diviser-pour-régner). Cependant, pour un code systématique, on préfère souvent une approche plus directe : on commence par construire le polynôme $D(X)$ de degré $k-1$ à partir du message, puis on calcule le reste $R(X) = D(X)X^r \bmod g(X)$ par une méthode utilisant la FFT.

5.2.2 Division polynomiale rapide par FFT

La division euclidienne de deux polynômes peut être accélérée en utilisant la transformée de Fourier. Étant donné $A(X)$ de degré $< n$ et $B(X)$ de degré r, on veut $Q(X)$ et $R(X)$ tels que $A = QB + R$ avec $\deg R < r$. En considérant les polynômes réversés, on peut montrer que $Q(X)$ est obtenu par une multiplication polynomiale rapide après une troncature. Concrètement, posons $A^*(X) = X^{\deg A} A(1/X)$ et $B^*(X) = X^{\deg B} B(1/X)$. Alors $Q^*(X) = A^*(X) \cdot (B^*(X))^{-1} \bmod X^{\deg A - \deg B + 1}$, où l'inverse est calculé modulo une puissance de X par l'algorithme de Newton utilisant la FFT. Cette approche permet d'obtenir $Q(X)$ en $O(n \log n)$. Le reste $R(X)$ s'en déduit par une multiplication polynomiale.

Ainsi, pour l'encodage, on prend $A(X) = D(X)X^r$, $B(X) = g(X)$, on calcule $Q(X)$ et $R(X)$ par cette méthode. Le mot de code est $C(X) = D(X)X^r + R(X)$.

Algorithm 5 Encodage systématique RS par FFT (division rapide)

Require: Message $\mathbf{d} \in \mathbb{F}_{2^q}^k$, polynôme générateur $g(X)$ de degré $r = n - k$, élément primitif α.

Ensure: Mot de code $\mathbf{c} = (p_0, \ldots, p_{r-1}, d_0, \ldots, d_{k-1})$.

1: Construire $D(X) = \sum_{i=0}^{k-1} d_i X^i$.
2: $A(X) \leftarrow D(X) \cdot X^r$ (simple décalage).
3: Calculer le quotient $Q(X)$ et le reste $R(X)$ de la division de $A(X)$ par $g(X)$ en utilisant la FFT pour l'inversion et la multiplication polynomiale.
4: Le polynôme du mot de code est $C(X) = A(X) + R(X)$.
5: Extraire les coefficients $p_0, \ldots, p_{r-1}$ de $R(X)$ et les d_i de $D(X)$.
6: **return** $\mathbf{c} = (p_0, \ldots, p_{r-1}, d_0, \ldots, d_{k-1})$.

5.3 Décodage systématique rapide par FFT

Le décodage par FFT suit les mêmes étapes que le décodage classique (syndromes, localisation, correction) mais chaque étape peut être accélérée.

5.3.1 Calcul rapide des syndromes

Soit $\mathbf{r}$ le mot reçu. Les syndromes sont $S_l = \sum_{i=0}^{n-1} r_i \alpha^{il}$ pour $l = 1, \ldots, 2t$ (en prenant $b = 1$). C'est exactement une transformée de Fourier de taille n appliquée à $\mathbf{r}$, mais on n'a besoin que de $2t$ valeurs. On peut calculer toutes les n valeurs par une FFT en $O(n \log n)$, ce qui est plus efficace que le calcul direct $O(nt)$ lorsque t est grand. Pour $n = 255$ et t quelconque, la FFT est avantageuse.

5.3.2 Localisation rapide des erreurs

L'algorithme de Berlekamp-Massey classique a une complexité $O(t^2)$. On peut le remplacer par une méthode utilisant la transformée de Fourier pour trouver les racines du polynôme localisateur. Une fois $\Lambda(X)$ connu (de degré ν), la recherche de Chien évalue Λ en n points α^{-i} pour localiser les erreurs. Cette évaluation multipoint peut être réalisée en $O(n \log^2 n)$ par un algorithme diviser-pour-régner utilisant la FFT. Cependant, pour des valeurs de n modérées, la recherche de Chien classique (itérative) est souvent suffisante. Néanmoins, si l'on souhaite optimiser, on peut utiliser une FFT pour évaluer le polynôme en tous les points d'un coup.

5.3.3 Calcul rapide des valeurs d'erreur

La formule de Forney nécessite l'évaluation de $\Omega(X)$ et $\Lambda'(X)$ aux points X_j^{-1}. Là encore, on peut utiliser une évaluation multipoint rapide.

Une approche globale plus élégante est l'algorithme de décodage par transformée de Fourier proposé par Blahut, qui utilise la transformée pour calculer les syndromes, puis pour trouver le polynôme localisateur par une méthode basée sur la transformée inverse. Nous donnons ici une version simplifiée.

Algorithm 6 Décodage RS par FFT (méthode de Blahut)

Require: Mot reçu $\mathbf{r} \in \mathbb{F}_{2^q}^n$, paramètres n, k, t, élément primitif α.
Ensure: Mot de code estimé $\hat{\mathbf{c}}$.

1: Calculer la FFT de $\mathbf{r}$: $R_l = \sum_{i=0}^{n-1} r_i \alpha^{il}$ pour $l = 0, \ldots, n-1$.
2: Les syndromes sont $S_l = R_l$ pour $l = 1, \ldots, 2t$. Si tous sont nuls, $\hat{\mathbf{c}} = \mathbf{r}$.
3: Sinon, construire le polynôme $S(X) = \sum_{l=1}^{2t} S_l X^{l-1}$.
4: Utiliser l'algorithme de Berlekamp-Massey (ou une variante par FFT) pour trouver $\Lambda(X)$.
5: Effectuer une recherche de Chien (ou une évaluation multipoint par FFT) pour obtenir les positions d'erreur $i_1, \ldots, i_\nu$.
6: Calculer $\Omega(X) = S(X)\Lambda(X) \bmod X^{2t}$.
7: Pour chaque position i_j, calculer $X_j = \alpha^{i_j}$, puis $Y_j = -\frac{X_j^{1-b}\Omega(X_j^{-1})}{\Lambda'(X_j^{-1})}$ (avec $b = 1$).
8: Corriger $\hat{r}_{i_j} = r_{i_j} - Y_j$.
9: **return** $\hat{\mathbf{c}}$.

5.4 Implémentation pratique de la NTT

Pour que la FFT soit efficace, on utilise généralement l'algorithme de Cooley-Tukey avec une factorisation de n en puissances de 2. Or, pour les codes RS classiques, $n = 2^m - 1$ n'est pas une puissance de deux. On peut alors :

— Utiliser une transformée de Winograd adaptée aux longueurs premières.

— Travailler dans une extension de corps où l'on dispose d'une racine d'ordre n (par exemple en prenant n comme un diviseur de $q-1$, ce qui est toujours le cas pour les codes RS primitifs, mais n n'est pas forcément une puissance de deux).

— Utiliser l'algorithme de Rader qui permet de calculer une DFT de taille première en la ramenant à une convolution cyclique, elle-même calculée par FFT.

Dans la pratique, pour $n = 255$, on peut utiliser une transformée de taille 256 en ajoutant un symbole fictif (zero-padding) et en adaptant les formules. Cela introduit une légère approximation mais reste acceptable si on traite les symboles supplémentaires avec soin.

Nous donnons ci-dessous un pseudo-code pour une NTT itérative lorsque n est une puissance de deux, ce qui peut servir de base pour des codes RS dont la longueur est adaptée.

Algorithm 7 NTT itérative (algorithme de Cooley-Tukey)

Require: Vecteur a de longueur n (puissance de 2), élément primitif ω d'ordre n (racine n-ième de l'unité).

Ensure: Vecteur $A = \mathrm{NTT}(a)$.

```
Effectuer le bit-reversal de a (réordonnancement des indices).
for len = 2, 4, 8, ..., n do
    wlen = ω^(n/len) (racine primitive d'ordre len)
    for i = 0 to n − 1 step len do
        w = 1
        for j = 0 to len/2 − 1 do
            u = a[i + j]
            v = a[i + j + len/2] · w
            a[i + j] = u + v
            a[i + j + len/2] = u − v      ▷ en caractéristique 2, soustraction = addition
            w = w · wlen
        end for
    end for
end for
return a
```

La transformée inverse s'obtient en utilisant ω^{-1} à la place de ω et en multipliant chaque coefficient par n^{-1} après la transformée.

5.5 Analyse de complexité et gains

Avec la NTT, l'encodage et le décodage passent d'une complexité quadratique à quasi-linéaire. Le tableau suivant résume les complexités comparées (pour un code de longueur n et t erreurs) :

Opération	Méthode classique	Méthode FFT
Encodage (division)	$O(nk)$	$O(n \log n)$
Calcul des syndromes	$O(nt)$	$O(n \log n)$
Berlekamp-Massey	$O(t^2)$	$O(t^2)$ (mais peut être amélioré)
Recherche de Chien	$O(n\nu)$	$O(n \log^2 n)$ (évaluation multipoint)
Forney	$O(\nu^2)$	$O(n \log^2 n)$

En pratique, pour $n = 255$ et t modéré, l'accélération la plus significative vient du calcul des syndromes et de l'encodage. De plus, les implémentations matérielles de la NTT bénéficient du parallélisme SIMD et des pipelines, permettant d'atteindre des débits très élevés.

L'utilisation de la transformée de Fourier rapide sur les corps finis constitue une amélioration majeure pour l'implémentation des codes de Reed-Solomon, en particulier dans les contextes nécessitant un débit élevé. Les algorithmes présentés permettent un encodage et un décodage systématiques avec une complexité quasi-linéaire. Les travaux fondateurs de Reed, Truong et Welch (1978) ont ouvert la voie, et depuis, de nombreuses optimisations (radix-8, corps de Fermat, transformées de Winograd) ont été développées pour traiter des codes très longs. L'intégration de ces techniques dans une bibliothèque comme AFF3CT pourrait offrir des gains de performance substantiels.

Références

[1] I. S. Reed et G. Solomon, *Polynomial Codes over Certain Finite Fields*, Journal of the Society for Industrial and Applied Mathematics, 1960.

[2] C. Leroux et al., *AFF3CT : A Fast Forward Error Correction Toolbox*, IEEE Open Journal of the Communications Society, 2020.

[3] S. Lin et D. J. Costello, *Error Control Coding*, Pearson Prentice Hall, 2004.

[4] S. B. Wicker et V. K. Bhargava, *Reed-Solomon Codes and Their Applications*, IEEE Press, 1995.

[5] E. R. Berlekamp, *Algebraic Coding Theory*, McGraw-Hill, 1968.

[6] F. J. MacWilliams et N. J. A. Sloane, *The Theory of Error-Correcting Codes*, North-Holland, 1977.

A Annexe : Tables de GF(8)

Avec polynôme primitif $p(x) = x^3 + x + 1$ et α racine de $p(x)$:

Table 1 – Représentation des éléments de GF(8)

Exponentielle	Polynomiale	Binaire
0	0	000
α^0	1	001
α^1	α	010
α^2	α^2	100
α^3	$\alpha + 1$	011
α^4	$\alpha^2 + \alpha$	110
α^5	$\alpha^2 + \alpha + 1$	111
α^6	$\alpha^2 + 1$	101